

StealthGuard IDS Project – Complete Explanation for Group Members

Project Title

StealthGuard – Intrusion Detection System (IDS)

1. Project Objective

The purpose of this project is to create a simple Intrusion Detection System (IDS) using:

- Bash Shell Scripting
- Python Programming
- Linux System Commands

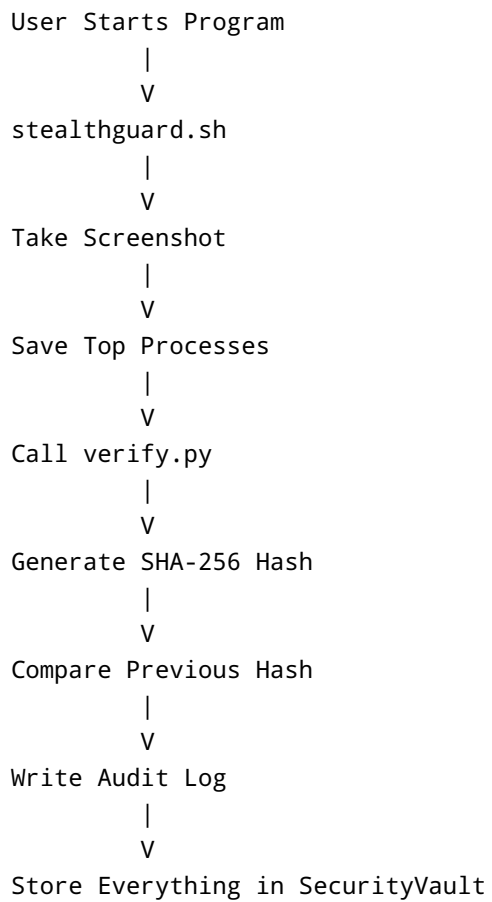
The system continuously monitors the computer screen and system activity by:

1. Taking screenshots
 2. Saving active process information
 3. Generating SHA-256 hashes
 4. Comparing screenshots
 5. Detecting changes
 6. Recording everything into an audit log
-

2. Software and Tools Used

Tool	Purpose
Kali Linux	Operating System
Bash Script	Automation
Python 3	Verification & Logging
Scrot	Screenshot Capturing
SHA-256	Integrity Checking
Nano Editor	Writing Code

3. Project Workflow



4. Folder Structure

```
stealthguard/
|
├─ stealthguard.sh
├─ verify.py
├─
└─ /SecurityVault
    |
    ├─ capture_xxx.png
    ├─ processes_xxx.txt
    ├─ system_security_audit.log
    └─ .previous_hash
```

5. Setup Procedure

Step 1: Update Package List

```
sudo apt update
```

Purpose

Updates Kali's package information before installing software.

Step 2: Install Scrot

```
sudo apt install scrot -y
```

Purpose

Scrot is a Linux screenshot utility.

Without Scrot, screenshots cannot be taken.

Step 3: Create Project Folder

```
mkdir ~/stealthguard  
cd ~/stealthguard
```

Purpose

Creates the project directory and moves inside it.

Step 4: Create Storage Vault

```
sudo mkdir /SecurityVault  
sudo chmod 777 /SecurityVault
```

Purpose

Creates a secure location where:

- Screenshots
- Logs
- Process files

will be stored.

chmod 777 Meaning

```
7 = Read + Write + Execute
```

Applied to:

- Owner
- Group
- Others

Therefore everyone can access the folder.

6. Creating the Shell Script

Create:

```
nano stealthguard.sh
```

Paste the Bash code and save.

7. Purpose of stealthguard.sh

This is the main controller of the project.

Responsibilities:

- Captures screenshots
- Saves process information
- Calls Python verifier
- Handles auto mode
- Handles manual mode

8. Explanation of Important Bash Code

Vault Location

```
VAULT="/SecurityVault"
```

Stores path of storage folder.

Log File

```
LOG="$VAULT/system_security_audit.log"
```

Stores location of audit log.

Verify Script Path

```
VERIFY_SCRIPT="$(dirname "$0")/verify.py"
```

Locates verify.py in the same folder.

Example

```
stealthguard/  
├─ stealthguard.sh  
└─ verify.py
```

Capture Interval

```
INTERVAL=10
```

Means automatic capture occurs every:

10 seconds

9. Terminal Color Variables

GREEN
RED
YELLOW
CYAN
NC

Used only for colorful output.

Example:

```
echo -e "${GREEN}[OK]${NC}"
```

Produces green text.

10. Vault Checking

```
if [ ! -d "$VAULT" ]
```

Checks whether SecurityVault exists.

If Missing

```
mkdir -p "$VAULT"
```

Creates it automatically.

11. Software Availability Check

Check Scrot

```
command -v scrot
```

Verifies screenshot software exists.

Check Python

```
command -v python3
```

Verifies Python is installed.

12. Main Capture Function

```
do_capture()
```

This function performs the entire monitoring operation.

13. Timestamp Generation

```
TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
```

Example:

```
2026-06-24_00-17-28
```

Purpose:

Makes every screenshot name unique.

14. Screenshot File Creation

```
FILENAME="$VAULT/capture_${TIMESTAMP}.png"
```

Example:

```
capture_2026-06-24_00-17-28.png
```

15. Screenshot Capture

```
scrot "$FILENAME"
```

Purpose:

Takes screenshot of current screen.

Stores it inside SecurityVault.

16. Checking Capture Success

```
if [ $? -eq 0 ]
```

Meaning

```
$? = Exit status of previous command  
0  = Success
```

If screenshot succeeded:

```
Screenshot saved
```

Otherwise:

Capture failed

17. Process Monitoring

```
ps aux --sort=-%cpu | head -11
```

ps aux

Displays all running processes.

Example:

```
Chrome  
Terminal  
VS Code  
System Services
```

--sort=-%cpu

Sorts processes by CPU usage.

Highest CPU consumer appears first.

Pipe Operator

```
|
```

Passes output to next command.

head -11

Displays:

- 1 Header Row
- Top 10 Processes

18. Process Snapshot Storage

```
processes_TIMESTAMP.txt
```

Example:

```
processes_2026-06-24_00-17-28.txt
```

Stores process list at that moment.

19. Calling Python Verifier

```
python3 verify.py screenshot processfile
```

Example:

```
python3 verify.py capture.png process.txt
```

Purpose:

Passes files for verification.

20. Creating verify.py

```
nano verify.py
```

Paste Python code and save.

21. Purpose of verify.py

Responsible for:

- Integrity verification

- SHA-256 generation
 - Log creation
 - Hash comparison
-

22. Python Libraries Used

os

Used for:

```
os.path.exists()  
os.path.getsize()
```

Checks file existence and size.

sys

Used for:

```
sys.argv
```

Reads command-line arguments.

hashlib

Used for:

```
hashlib.sha256()
```

Creates SHA-256 hashes.

datetime

Used for:

```
datetime.now()
```

Generates timestamps.

23. SHA-256 Hashing

Example:

```
66362731d9aaf9ab57a8...
```

Purpose:

Creates unique fingerprint of file.

Even a tiny change creates a completely different hash.

24. Verification Process

The Python script performs 3 checks.

Check 1: File Exists

```
os.path.exists()
```

Verifies screenshot is present.

Failure:

```
File not found
```

Check 2: File Size

```
os.path.getsize()
```

Checks whether screenshot is empty.

Failure:

```
Zero-byte file
```

Check 3: SHA-256 Integrity Check

```
current_hash=get_sha256()
```

Generates hash.

25. Previous Hash Comparison

Python reads:

```
.previous_hash
```

First Capture

No previous hash exists.

Output:

```
FIRST RUN
```

Same Hash

Current hash = Previous hash

Output:

```
NO CHANGE
```

Different Hash

Current hash $\neq$ Previous hash

Output:

```
ALERT - Screen change detected
```

26. Why Did ALERT Appear in Our Demo?

During testing:

First Hash:

```
66362731d9aaf9ab57a8...
```

Second Hash:

```
c705d3b533c50bb91c45...
```

Both hashes were different.

Therefore:

```
ALERT - Screen change detected
```

was generated.

Reason:

The screen changed between captures.

Even:

- Cursor movement
- New terminal output
- Window movement

changes screenshot pixels.

27. Audit Log Creation

Stored in:

```
system_security_audit.log
```

Contains:

- Screenshot name
- File size
- SHA-256 hash
- Status
- Process list

Example:

```
CAPTURE  
SHA-256  
Status  
Processes
```

28. Manual Mode

Command:

```
./stealthguard.sh --capture
```

Purpose:

Takes one screenshot and exits.

29. Automatic Mode

Command:

```
./stealthguard.sh
```

Purpose:

Runs continuously.

Every 10 seconds:

1. Screenshot taken
2. Processes recorded
3. Hash generated
4. Audit log updated

Stop using:

```
Ctrl + C
```

30. Results Obtained

Successfully achieved:

- ✓ Screenshot Capture
 - ✓ Process Monitoring
 - ✓ SHA-256 Generation
 - ✓ Integrity Verification
 - ✓ Hash Comparison
 - ✓ Change Detection
 - ✓ Audit Logging
 - ✓ Manual Mode
 - ✓ Automatic Mode
-

31. Viva Questions and Answers

What is IDS?

Intrusion Detection System.

A security tool that monitors activities and detects suspicious changes.

Why SHA-256?

Used for file integrity verification.

Different content produces different hashes.

Why compare hashes?

To determine whether screen contents changed.

Why save process information?

To identify active applications and suspicious processes.

Why use Bash?

Good for Linux automation and command execution.

Why use Python?

Provides easier implementation of hashing, verification, and logging.

What is the role of `.previous_hash`?

Stores last screenshot hash for comparison.

What happens when hashes differ?

System generates:

ALERT - Screen change detected

and records it in audit logs.

Conclusion

StealthGuard successfully combines Bash scripting and Python programming to create a lightweight Intrusion Detection System. The system captures screenshots, monitors active processes, verifies screenshot integrity using SHA-256 hashing, detects changes through hash comparison, and records all activities in an audit log for security monitoring and analysis.